

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LF09:19:Von der Restschnittstelle zu HTTP - eine Top-Down-Annäherung

(1) Ausgang

Nehmen wir an, wir

- hätten eine sensorische Überwachung mit einem
 - Sensor 1 in Raum 1, der die dortige Temperatur und Luftdruck (Hauptsensor) liefert
 - Sensor 2 in Raum 1, der die dortige Lautstärke (Nebensensor)
 - Sensor 1 in Raum 2, der die dortige Temperatur und Luftdruck (Hauptsensor) liefert
 - Sensor 2 in Raum 2, der die dortige Luftfeuchtigkeit (Nebensensor) liefert
- hätten ein Datenhaltungssystem (z.B. eine Datenbank), in dem wir die Messungen aufbewahren
- und sollten als Server Anfragen nach Werten, Datensätzen und Datenausschnitte bereitstellen.

Lösungsansätze:

- **SQL-Nativ**
 - Client formuliert erstellt SQL-Anfrage.
 - Client schickte Anfrage zu SQL-Server.
 - SQL-Server holt Daten aus der DB.
 - SQL-Server schickt Daten als Antwort.
 - Client parst Ergebnis.
 - Client tut Datenbasierte seine Arbeit.
 - *Vorteil:* Standardmethode
 - *Nachteil:* Aufwendige Vor- und Nachberechnung auf Clientseite.
- **Webapplikation**
 - Client füllt HTML-Formular aus
 - Client schickt Daten per HTTP an Webapplikation (=Webserver mit aktivierten Php-/Python-Modul und passenden Php/Python-Serverskript)
 - Webapplikation sucht Daten (aus Datenbank oder Filesystem) zusammen
 - Webapplikation liefert Daten als HTTP-Response JSON-Datei zurück

- *Vorteil:* Weniger Vor- und Nachberechnung auf Clientseite
- *Nachteil:* Aufwendige Anfrageanalysen auf Serverseite.

- **REST-Schnittstelle**

- Client ruft nur und direkt eine 'Datenart spezifische' URL (Resource) auf
- Fertig durch programmierte 'Datenart spezifische' Webapplikation stellt JSON-Antwort zusammen
- Webapplikation liefert Daten als HTTP JSON-Datei zurück
- *Vorteil:* Weniger Vor- und Nachberechnung auf Client- und auf Serverseite
- *Vorteil:* Anfrageparametrisierung in 'Ordnernamen' parametrisiert
- *Nachteil:* Mehrere Wepapplikationsskripte zu pflegen

(2) Beispiel A

Angenommen, wir wollten folgende Anfragen ermöglichen:

1. (Alle) Temperaturmessungen aus allen Räumen
2. (Alle) Temperatur- und Luftdruckmessungen aus allen Räumen
3. (Alle) Messungen aus Raum 1
4. (Alle) Messungen aus Raum 2

Man sieht: Die ausgelieferten Messungen müssen nicht mit den Topics

- R1/S1/Temp_Press
- R1/S2/Volume
- R2/S1/Temp_Press
- R2/S2/Humidity

der MQTT basierten Datenanlieferungen in das Datenhaltungssystem übereinstimmen:

Die 'Datenart spezifische' Webapplikation berechnet bildet die Daten auf die bestimmungsgemäße HTTP-Resonse ab.

Dann bräuchten wir in unserem HTTP-Server folgende Datenstruktur:

```
1 | -raum-1
2 | -raum-2
3 | -raum-a
4 |   -t
5 |   -tup
```

- Übersicht: <http://127.0.0.1/dwa/>
- Temperaturmessungen aus allen Räumen: <http://127.0.0.1/dwa/raum-a/t/>

- Temperatur- und Luftdruckmessungen aus allen Räumen: <http://127.0.0.1/dwa/raum-a/tup/>
- Messungen aus Raum 1: <http://127.0.0.1/dwa/raum-1>
- Messungen aus Raum 2: <http://127.0.0.1/dwa/raum-2>

(3) REST

- steht für *Representational State Transfer*
- dient bevorzugt für Maschine-zu-Maschine-Kommunikation
- “kodiert keine Methodeninformation in den URI, da ein URI nur den Ort (URL) oder Namen (URN) einer Ressource angibt, nicht aber die Funktionalität, die der Web-Dienst zu der Ressource anbietet”
- verlangt, dass weder der Server noch die Anwendung Zustandsinformationen zwischen zwei Nachrichten speichern sollen (**stateless**)

Abgrenzung:

- Onlinedienst mit nur statischen Seiten: REST-Konform
- Nachrichtenseiten mit sich ständig ändernde Informationen mit sowohl unterschiedlichem Format als auch Inhalt an: NICHT REST-Konform
- Webseite, auf der ständig die aktuelle Uhrzeit in immer demselben Format abrufbar ist: REST-Konform
- https://de.wikipedia.org/wiki/Representational_State_Transfer
- <https://en.wikipedia.org/wiki/REST>

Weiterentwicklung: von REST zu RESTful

“RESTful bezeichnet folgenden Prozess: Ein Client fordert eine Ressource über eine herkömmlich formatierte URL (RESTful-API) an und erhält eine herkömmlich formatierte JSON-Antwort zurück, die die Ressource mithilfe von Key/Value-Paaren beschreibt. Das sind die charakteristischen Merkmale moderner RESTful-Webanwendungen. Davei

- beschreibt der URL-Pfad, wo ein Asset vorhanden ist, während
 - das HTTP-Verb beschreibt, wie es verändert werden kann.
- <https://www.computerwoche.de/article/2827943/was-ist-rest.html>

(4) http

- steht für *Hypertext Transfer Protocol*

- mit HTML zusammen von Sir Tim Berners-Lee entworfen/erfunden
- ist ein 1991 eingeführtes zustandsloses Protokoll zur Übertragung von Daten auf der Anwendungsschicht über ein Rechnernetz
- wird hauptsächlich eingesetzt, um Webseiten (Hypertext-Dokumente) aus dem World Wide Web (WWW) in einen Webbrowser zu laden
- nicht darauf eingeschränkt
- setzt eine Client-Server-Architektur voraus
- beide tauschen Nachrichten aus:
 - Client-Nachrichten an den Server heißen *Request* (Anfragen)
 - Server-Nachrichten an den Client heißen *Response* (Anfragen)
- jede Nachricht besteht aus einem HEADER und der eigentlichen Message, dem BODY
- HTTP/1.0 (1996)
 - baut vor jeder Anfrage eine neue TCP/IP Verbindung auf
- HTTP/1.1 (1999)
 - erlaubt Keepalive
 - erlaubt als Methode auch PUT
- HTTP/2 (2015)
 - erlaubt Bündeln von Anfragen
 - erweiterte Kodierung
 - erweiterte Übertragung von Binärdateien
- HTTP/3 (2022)
 - Beschleunigung durch UDP-basierte Übertragung.

Methoden:

- **GET** : Methode zum Anfordern einer Ressource
- **POST** : Methode zur Übermittlung von zu verarbeitenden Daten
- **HEAD** : Methode zur Anfrage des Antwortheaders ohne Body
- **PUT** : Methode zur Übermittlung von serverseitig zu speichernden Daten
- **PATCH** : Methode zur Änderung von Teilen von serverseitig existierenden Dateien
- **DELETE** : Methode die Löschung einer serverseitig existierende Datei anzufordern
- **TRACE** : Methode die eigene Anfrage wie serverseitig angekommen abzufragen
- **OPTIONS** : Methode, die vom Server unterstützten Methoden und Merkmale zu erfahren.
- **CONNECT**: (für Proxyserver)

Hinweis:

Die Nutzung der Server hängt davon ab

- ob der Server die Protokollspezifikation implementiert hat
- ob der Server so konfiguriert ist, dass die Nutzung Client seitig zugelassen ist

(6) Beispiel B

Gesetzt, wir hätten folgendes simples PHP-Skript auf unserem Webserver laufen:

```
1 $USER=htmlspecialchars($_GET["user"]);
2 print("<html>\n<head>\n</head>\n<body>\n<p>");
3 print("Nachricht von $USER angekommen");
4 print("</p>\n</body>\n</html>");
```

Dann könnten wir es

- 1.) mit `http://127.0.0.1/auth.sim.php?user=karsten` aus dem Browser heraus aufrufen
- 2.) nach Eingabe des Shell-Kommandos `>telnet 127.0.0.1 80` mit diesen Zeilen aufrufen:

```
1 GET /auth.sim.php?user=karsten HTTP/1.1
2 HOST: localhost
```

- 3.) nach Eingabe des Shell-Kommandos `>telnet 127.0.0.1 80` mit diesen Zeilen aufrufen:

```
1 POST /auth.sim.php HTTP/1.1
2 HOST: localhost
3 Content-Type: application/x-www-form-urlencoded
4 CONTENT-LENGTH: 14
5
6 user=karsten
```

- 4.) nach Eingabe des Shell-Kommandos `>telnet 127.0.0.1 80` mit diesen Zeilen aufrufen:

```
1 HEAD /auth.sim.php HTTP/1.1
2 HOST: localhost
```

Anmerkung zu Telnet:

- ist ein Client-Server basiertes Protokoll zum Austausch von Nachrichten.
- erlaubt Shell-Zugriff auf den Server
- wird heute nicht mehr benutzt, weil alle Informationen im Klartext übers Netz gehen, insbesondere auch Passwörter!

- Nachteil ist Vorteil:
 - Client baut TCP/IP zum Server auf
 - Überträgt allen Inhalt 1:1 zum Server
 - kann deshalb als manuelle Übertragungsschnittstelle für andere TCP/IP basierte Protokolle genutzt werden.
 - https://de.wikipedia.org/wiki/Hypertext_Transfer_Protocol
 - <https://wiki.selfhtml.org/wiki/HTTP/Anfragemethoden>
-

ÜBUNG LF09:19:http-rest:01

Ihnen steht wieder der Rechner mit ssh-Server aus den vergangenen Stunden zur Verfügung:

- einen Ubuntu 25.04 Serverrechner mit
 - einem User `gsldkm` mit Passwort `.gsldkPw4all` und
 - einem User `root` mit Passwort `lgsldkPw4all`
- und für das Schul-Wlan einen WLAN-User `fius` mit Passwort `fachinformatikeruser`.

Auf dem Rechner läuft auch ein Apache2.0-Webservers mit HTTP-Root `/var/www/html`.

- ☐ Konfigurieren und/oder bauen Sie den Rechner so um, dass er jetzt auch die HTTP-Befehle `PUT` und `DELETE` akzeptiert und umsetzt.
- ☐ Lassen Sie per HTTP-Befehl vom Client aus einen nach Ihrer Klasse benannten Ordner anlegen.
- ☐ Legen Sie jeder eine nach Ihnen benannte Datei in dem Ordner ab.

Lösung:

- 1. Check, ob Zugriff auf `/var/www/html` mittels Browser `http://IPADRESSE/index.html`.
 - 2. Ein CGI-Skript (in Python) schreiben, dass die Befehle `PUT` und `Delete` umsetzt.
 - 3. Apache so konfigurieren, dass er `PUT` und `DELETE`-Befehle an das CGI-Skript umlenkt.
-